

Title

- Guest lecture by Jason Choi (UC Berkeley, Hybrid Robotics) in Sylvia Herbert's safe-control course at UCSD, Feb 11, 2021.
- Two tools for controllers with guarantees: a **CLF** (control Lyapunov function) makes the system settle at a goal; a **CBF** (control barrier function) keeps it inside a safe region.
- Both become a small optimization solved at every time step, so they run fast enough for real robots.



Motivation 1: emergency braking

- Dashcam video: a car's automatic emergency braking stops for a pedestrian who steps out suddenly at night.
- Lesson: a safety controller must react within a fraction of a second, so it has to be cheap to compute.

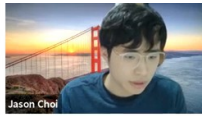




2021-02-11 12:35:40

Motivation 2: owl in a tight space

- An owl flies through a narrow gap between people's legs to reach its target (a gloved hand) without touching anyone.
- Lesson: animals stay safe in cramped spaces even though their dynamics are highly nonlinear and high-dimensional. We want robots to do the same.
- Goal of the lecture: safety controllers that are fast *and* handle nonlinear, high-dimensional systems.

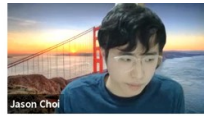


Contents

- **Backgrounds**
  - Lyapunov Theory, Invariance Principle
  - Control Affine System
  - Control Lyapunov Function (CLF) and CLF-QP
  - Control Barrier Function (CBF) and CBF-CLF-QP
- **Design Steps (Toy example – Adaptive Cruise Control)**
- **How to code up? (Toy example – Adaptive Cruise Control)**
- **Comparison with HJ Reachability**
- **Research Challenges**

Outline

- Background: Lyapunov theory, Nagumo's invariance principle, control-affine systems.
- CLF and CLF-QP; CBF and CBF-CLF-QP. A QP (quadratic program) is a small optimization that solvers finish in well under a millisecond.
- Worked example: adaptive cruise control (design steps and MATLAB code).
- Comparison with Hamilton–Jacobi (HJ) reachability, then open research problems.



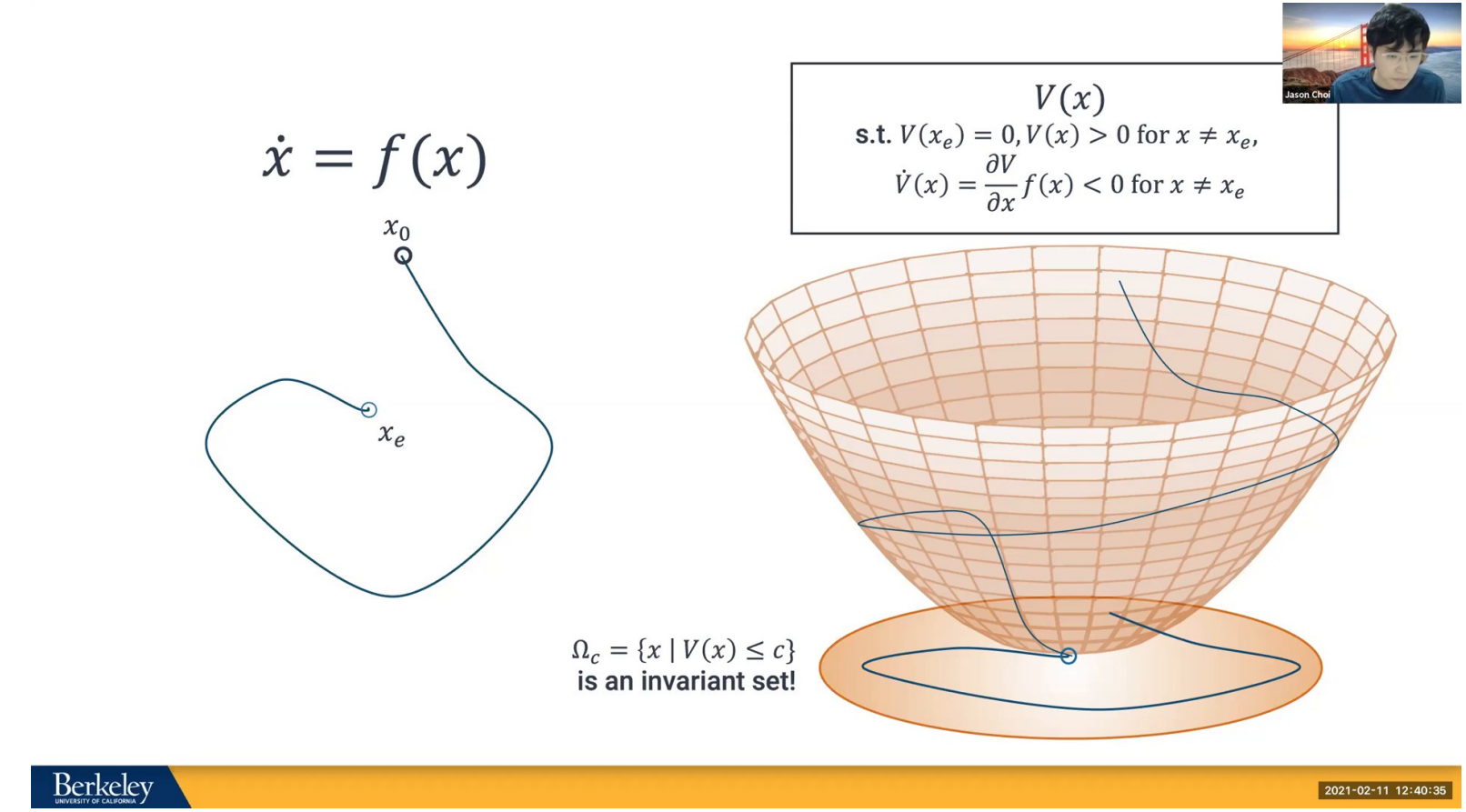
Lyapunov Theory

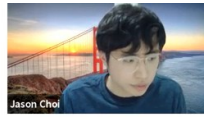
Section: Lyapunov theory

- Idea: invent an "energy" function of the state. If the energy always goes down, the system has to settle at its resting point.

Lyapunov function

- System with no control: $\dot{x} = f(x)$. Question: do trajectories reach the equilibrium x_e?
- Find $V(x)$ with $V(x_e) = 0$, $V > 0$ elsewhere, and $\dot{V} = \frac{\partial V}{\partial x} f(x) < 0$ for $x \neq x_e$. Then V shrinks to 0 along every trajectory, so the system is stable. No need to solve the ODE.
- Bonus: every sublevel set $\Omega_c = \{x : V(x) \leq c\}$ is **invariant** (start inside, stay inside), because V can only go down.

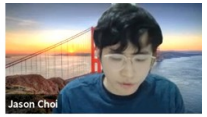




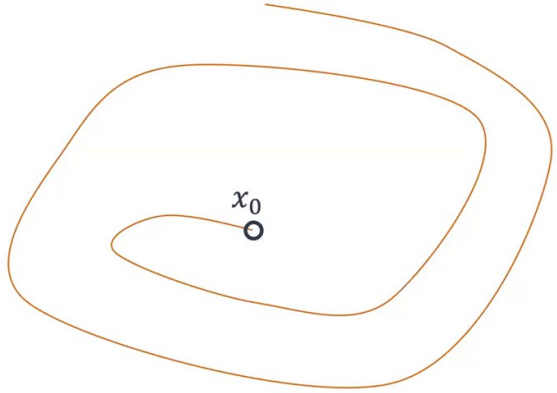
Nagumo's Invariance Principle

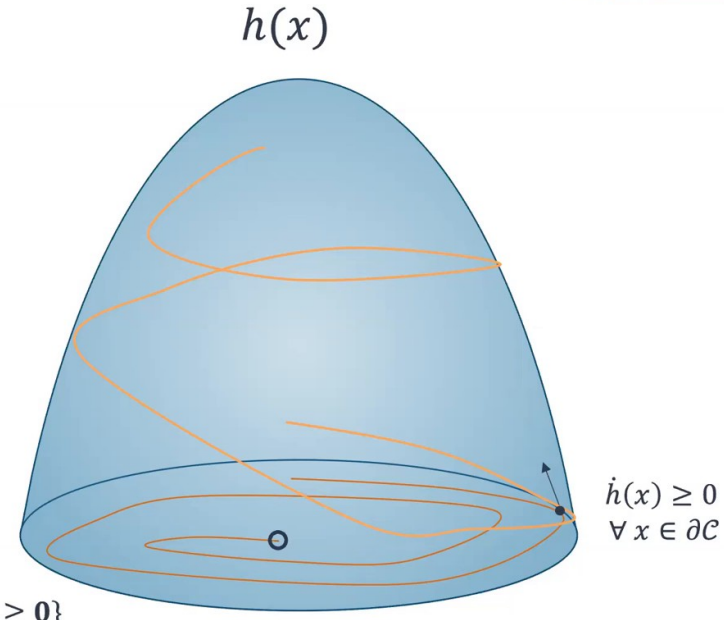
Section: Nagumo's principle

- Second building block, used later for CBFs. It looks like the Lyapunov idea but asks a different question: does the trajectory *stay inside a region*, not does it reach a point?



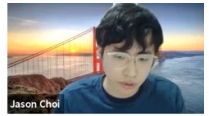
$\dot{x} = f(x)$





Nagumo's invariance principle

- Safe set $\mathcal{C} = \{x : h(x) \geq 0\}$. Picture h as an upside-down Lyapunov function.
- Condition only on the edge: $\dot{h} \geq 0$ wherever $h = 0$. Inside the set, h may rise or fall freely.
- A trajectory that touches the edge is pushed back in, so $\mathcal{C}$ is invariant.
- Difference from Lyapunov: only the zero level set is guaranteed invariant, not every level set. That makes it much less restrictive.



Dynamics – Control Affine System

Expression for dynamics of a general nonlinear controlled system:

$$\dot{x} = F(x, \boldsymbol{u})$$

where $x \in \mathbb{R}^n$ is the system state, $u \in \mathbb{R}^m$ is the control input.

If F is Lipschitz continuous in x, continuous in u, and if $u(\cdot)$ is piecewise continuous in t, we are guaranteed that given an initial state $x(t_0) = x_0$, the trajectory of the dynamics $x(t)$ exists and it is unique.

Stability → Stabilizability

(Forward) Invariant → Control Invariant

Adding control

- General controlled system: $\dot{x} = F(x, u)$, state $x \in \mathbb{R}^n$, input $u \in \mathbb{R}^m$.
- If F is Lipschitz in x, continuous in u, and $u(t)$ is piecewise continuous, each start state plus input signal gives exactly one trajectory.
- With control, the questions change: stability $\rightarrow$ **stabilizability** (can *some* u make it stable?) and invariance $\rightarrow$ **control invariance** (can some u keep it inside the set?).



Dynamics – Control Affine System

We will mainly deal with a specific type of a nonlinear system, a control affine system:

$$\dot{x} = \underbrace{f(x)}_{\text{"drift term"}} + \underbrace{g(x)u}_{\text{"actuation effect"}}$$

"autonomous vector field" "control vector field"

where $f: \mathbb{R}^n \to \mathbb{R}^n$ and $g: \mathbb{R}^n \to \mathbb{R}^{n \times m}$ are Lipschitz continuous in x.

Many mechanical systems are control affine systems:

$$\begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ D^{-1}(-C\dot{q} - G) \end{bmatrix} + \begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix} u$$

Control-affine systems

- From here on: $\dot{x} = f(x) + g(x)u$. f = drift (what happens with zero input); $g(x)u$ = input effect, linear in u.
- Why: at a fixed state the input enters linearly, which later turns the controller into a QP.
- Covers most robots. Lagrangian mechanics with state $(q, \dot{q})$ gives $\ddot{q} = D^{-1}(-C\dot{q} - G) + D^{-1}B\,u$.



Control Lyapunov Function (CLF)

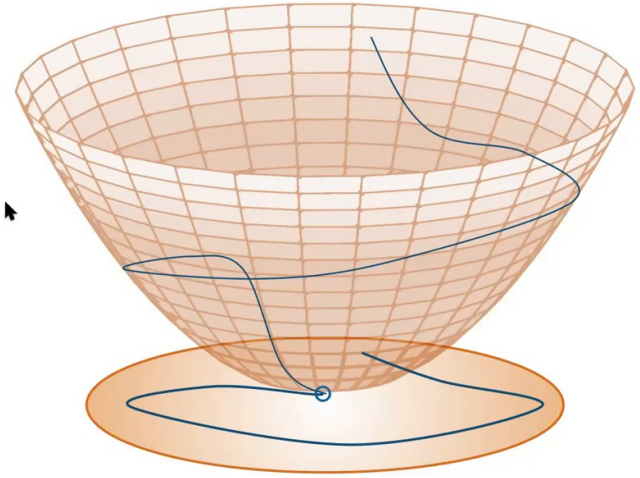
Recall the definition of the Lyapunov function.

$$\dot{x} = F(x, u)$$

$$V(x)$$

s.t. $V(x_e) = 0, V(x) > 0$ for $x \neq x_e$,

$\exists u$, s.t. $\dot{V}(x, u) < 0$ for $x \neq x_e$



CLF: the idea

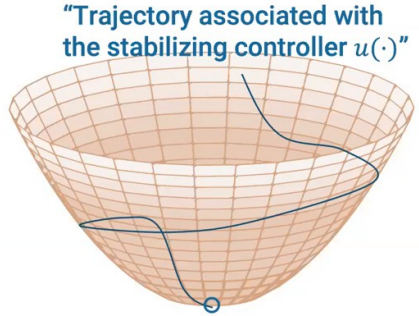
- Same conditions as a Lyapunov function, except the last becomes: *there exists* an input u with $\dot{V}(x, u) < 0$.
- In words: V need not fall on its own; you just need to be able to steer so that it falls.



Control Lyapunov Function (CLF)

Let $V(x)$: $\mathbb{R}^n \to \mathbb{R}$ be a continuously differentiable function.
If there exists a constant $c > 0$ such that

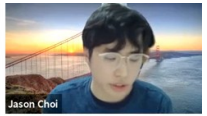
1) $\Omega_c := \{x \in \mathbb{R}^n : V(x) \leq c\}$, a sublevel set of $V(x)$ is bounded,
2) $V(x) > 0$ for all $s \in \mathbb{R}^n \backslash \{x_e\}$, $\quad V(x_e) = 0$,
3) $\min_{u \in U} \dot{V}(x,u) < 0$ for all $x \in \Omega_c \backslash \{x_e\}$,



Then $V(x)$ is a (local) **Control Lyapunov Function (CLF)**
and Ω_c is a region of attraction (ROA), i.e.
every state in Ω_c is asymptotically stabilizable to x_e.
$(\forall x_0 \in \Omega_c,\ \text{for } x(t) \text{ with } x(0) = x_0,\ \exists u: [0,t] \to U \text{ s.t. } \lim_{t \to \infty} x(t) = x_e)$

CLF: formal definition

- V is a (local) CLF if, for some $c > 0$: (1) $\Omega_c = \{V \leq c\}$ is bounded, (2) $V > 0$ except $V(x_e) = 0$, (3) $\min_{u \in U} \dot{V}(x,u) < 0$ on $\Omega_c \setminus \{x_e\}$.
- Then Ω_c is a **region of attraction**: from any start inside, some input drives $x(t) \to x_e$.
- Q&A: Ω_c *under*-estimates the set you can really stabilize from (HJ's backward reachable tube contains it). A quadratic V gives ellipse-shaped sets; an HJ-built V could cover everything but is hard to compute.
- This is only *asymptotic*: $x \to x_e$ as $t \to \infty$, with no promise on speed.



Control Lyapunov Function (CLF)

Derivative of $V(x)$ along the dynamics:

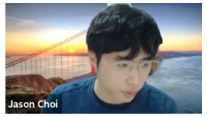
$$\begin{aligned}\dot{V}(x,u) &= \nabla V(x)\cdot\dot{x}\\ &= \nabla V(x)\cdot f(x) + \nabla V(x)\cdot g(x)u\\ &= \boldsymbol{L_f V(x) + L_g V(x) u}\end{aligned}$$

($L_p q(x) \coloneqq \nabla q(x)\cdot p(x)$ is a Lie derivative operator used to make formulas concise.)

is affine in u.

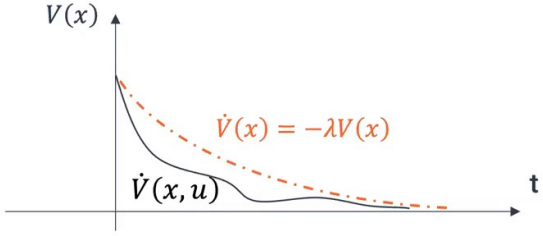
$\dot{V}$ is linear in u

- For control-affine dynamics: $\dot{V} = \nabla V \cdot f + \nabla V \cdot g\, u = L_f V(x) + L_g V(x)\, u.$
- $L_f V$ and $L_g V$ ("Lie derivatives") are shorthand for those two dot products.
- Key fact: at a given state, $\dot{V}$ is affine in u. Example: if $L_f V = 2$ and $L_g V = -1$, then $\dot{V} = 2 - u$, so any $u > 2$ makes V fall.



Exponentially Stabilizing Control Lyapunov Function (CLF):

- **In Asymptotic stability, there's no notion of "how fast" the system will reach the equilibrium.**
- **We can go one step further and construct a Lyapunov constraint that impose a condition of minimum rate of decrease in $V(x)$.**

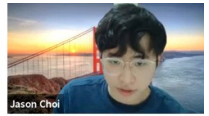


*Aaron Ames, et al., TAC 2014

▶ 22:12 also ▶ 22:39

Exponential CLF: why

- Asymptotic stability says nothing about *how fast* you reach the equilibrium.
- Fix: make V fall at least as fast as the curve $\dot{V} = -\lambda V$, i.e. $V(t) \leq V(0)\, e^{-\lambda t}$.

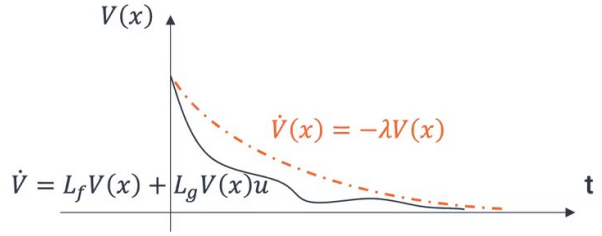


Exponentially Stabilizing Control Lyapunov Function (CLF):

Let $V(x)$: $\mathbb{R}^n \to \mathbb{R}$ be a continuously differentiable, positive definite, and radially unbounded function. If there exists some constant $\lambda > 0$ such that

$$\min_{u \in U} \dot{V}(x,u) + \lambda V(x) \leq 0,$$

then $V(x)$ is an exponentially stabilizing CLF (ES-CLF) and any $x \in \mathbb{R}^n$ is exponentially stabilizable to x_e*. λ serves as a decay rate of an upper bound of $V(x(t))$.



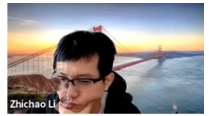
$$\dot{V}(x,u) + \lambda V(x) \leq 0, \text{ for some } u \in U, \lambda > 0$$

*Aaron Ames, et al., TAC 2014

2021-02-11 12:57:37

Exponential CLF: definition

- **ES-CLF**: V is positive definite and radially unbounded, and for some $\lambda > 0$ there is a u with $\dot{V}(x,u) + \lambda V(x) \leq 0$.
- λ is the guaranteed decay rate. Example: $\lambda = 2$ halves the bound on V every $\ln 2/2 \approx 0.35$ s.
- This single inequality, $L_f V + L_g V\, u + \lambda V \leq 0$, is what the controller will enforce.



CLF-QP

- **The CLF constraint is linear in u, so using it as a constraint in a min-norm controller results in a Quadratic Program (QP) formulation.**

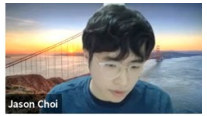
$$\underset{u,\,\delta}{\operatorname{argmin}} \quad (u - u_{ref})^T H (u - u_{ref}) + p\delta^2$$

u: control input
δ: slack variable

$$\text{subject to: } L_f V(x) + L_g V(x) u + \lambda V(x) \leq \delta \quad \text{CLF Constraint}$$
$$u \in U \quad \text{Input Constraints}$$

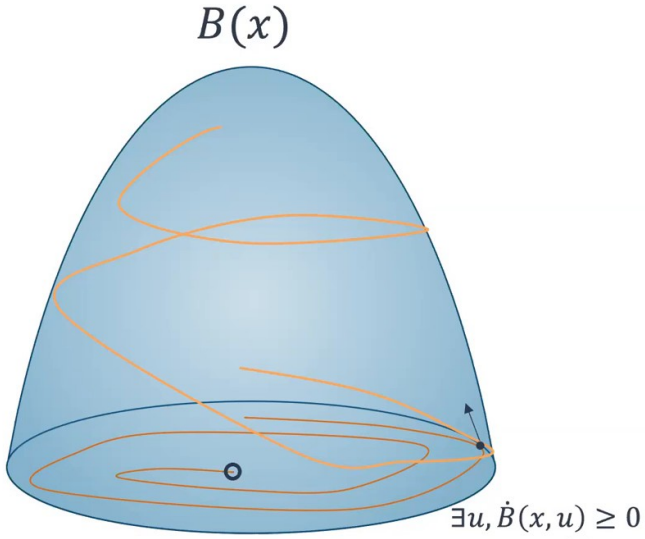
CLF-QP controller

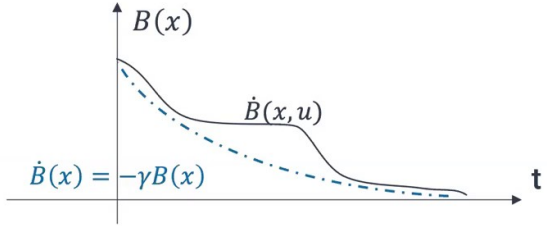
- Each time step solve:
  $\min_u (u - u_{\text{ref}})^\top H (u - u_{\text{ref}}) + p\,\delta^2$ s.t.
  $L_f V + L_g V\, u + \lambda V \leq \delta,\ u \in U.$
- The constraint is linear in u, so this is a QP. It runs at a few hundred Hz on walking robots.
- u_{ref} = any nominal command (zero, steady-state input, planner, learned policy). The QP returns the closest input that still meets the constraint.
- Slack δ softens the constraint so the QP stays solvable when V isn't a verified CLF. $\delta > 0$ means the guarantee is given up for that step.
- Q&A: without control-affine form the constraint is nonlinear, which is too slow online. Unlike HJ (push V down as fast as possible), this only needs V to fall *enough*, using the smallest input. u_{ref} changes the answer but not the set of allowed inputs.



Control Barrier Function (CBF)

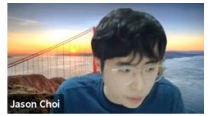
Recall Nagumo's invariance principle.





CBF: the idea

- Start from Nagumo: need some u with $\dot{B} \geq 0$ on the edge of $\mathcal{C} = \{B \geq 0\}$.
- Strengthen it: keep $B(t)$ above the curve $\dot{B} = -\gamma B$. That curve approaches 0 but never reaches it, so B never goes negative and you never leave the safe set.



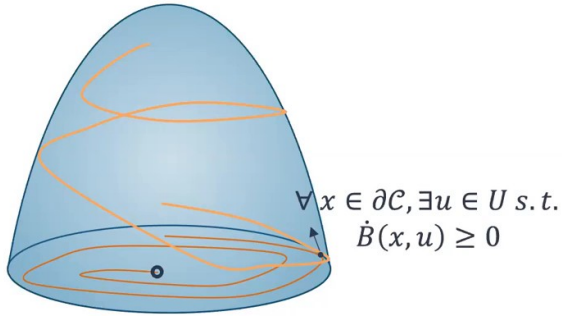
Control Barrier Function (CBF)

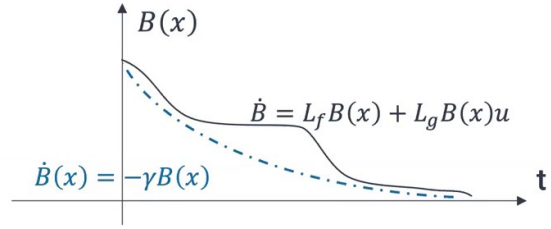
Let $B(x): \mathbb{R}^n \to \mathbb{R}$ a continuously differentiable function whose zero-superlevel set is $\mathcal{C}$ ($\mathcal{C} = \{x \mid B(x) \geq 0\}$) and $\nabla B(x) \neq 0$ for all $x \in \partial\mathcal{C}$.

If there exists a positive coefficient γ such that for all $x \in \mathcal{C}$

$$\sup_{u \in U} [L_f B(x) + L_g B(x) u] + \gamma B(x) \geq 0,$$

then $B(x)$ is a Control Barrier Function and any Lipschitz continuous control law that satisfies the above constraint will render the set $\mathcal{C}$ safe (i.e. control invariant)*.

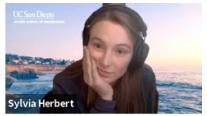




$$\dot{B}(x,u) + \gamma B(x) \geq 0, \text{ for some } u \in U, \; \gamma > 0$$

CBF: formal definition

- B is a CBF if $\nabla B \neq 0$ on $\partial\mathcal{C}$ and $\sup_{u \in U} [L_f B + L_g B\, u] + \gamma B \geq 0$ for all $x \in \mathcal{C}$.
- Then any Lipschitz controller satisfying $\dot{B} + \gamma B \geq 0$ keeps $\mathcal{C}$ safe (control invariant). It mirrors the CLF with "≥ 0" in place of "≤ 0".
- Q&A: why add γB instead of just "$\dot{B} \geq 0$ on the edge"? (1) The car slows smoothly as it nears the edge instead of bouncing off it. (2) It is more forgiving when B is hand-designed. (3) It matches the CLF form. (4) It is far less strict than making every level set invariant, since B may fall when far from the edge.



CBF-CLF-QP

$$\underset{\substack{u\text{: control input}\\ \delta\text{: slack variable}}}{\operatorname{argmin}} \quad (u-u_{ref})^T H(u-u_{ref}) + p\delta^2$$

subject to: $L_f V(x) + L_g V(x)u + \lambda V(x) \leq \delta$ — CLF Constraint

$L_f B(x) + L_g B(x)u + \gamma B(x) \geq 0$ — CBF Constraint

$u \in U$ — Input Constraint

CBF-CLF-QP

- Same cost, plus the CBF constraint $L_f B + L_g B\, u + \gamma B \geq 0$ as a *hard* constraint. The CLF stays *soft* (slack δ).
- So safety beats tracking. If both can't hold, δ grows and stability gives way. If both hold, $\delta = 0$.
- If B was built with the input limits in mind, the QP is always feasible; in practice that is often skipped.
- Several obstacles: add one constraint per B_i. Careful: they can conflict (dodging one pushes you into another) and make the QP infeasible.



CBF-CLF Helper

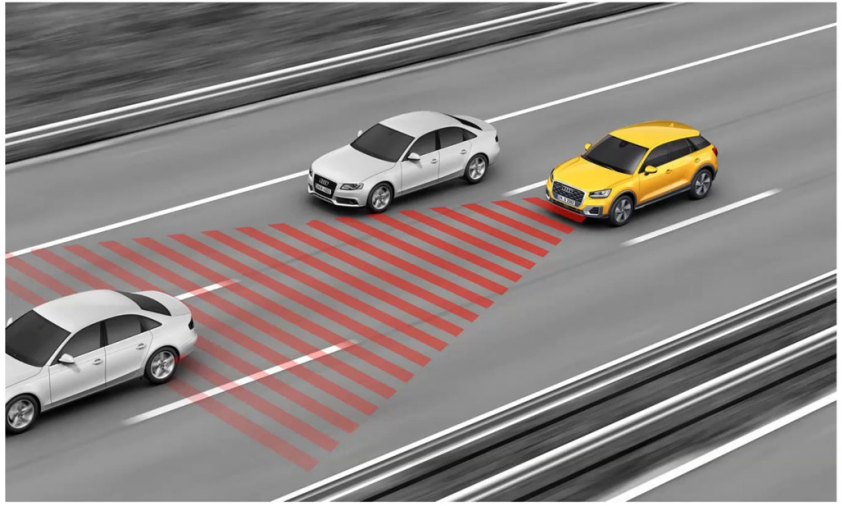
- Library: https://github.com/HybridRobotics/CBF-CLF-Helper
- Designed to let users easily implement safety-controller based on CBFs and CLFs with Matlab.
  - An easy interface for construction and simulation of a control-affine system.
  - Safety controller including CLF-QP, CBF-QP, and CBF-CLF-QP as built-in functions.

CBF-CLF-Helper library

- MATLAB library (HybridRobotics/CBF-CLF-Helper) for defining control-affine systems and running CLF-QP, CBF-QP and CBF-CLF-QP.
- The live code walkthrough was skipped for time. The repo's manual has the full derivations.



Adaptive Cruise Control

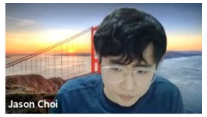


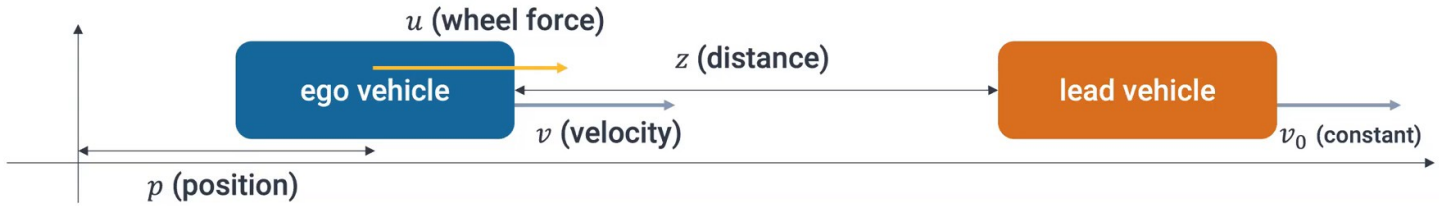
- Control objective: Pursuing a desired velocity (CLF) while maintaining safety distance with a lead vehicle (CBF).
- This example is excerpted from A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," CDC 2014, with a slight modification.

Example: adaptive cruise control

- From Ames, Grizzle & Tabuada (CDC 2014), the paper that introduced CBF-based QPs.
- Goal: drive at the driver's set speed (CLF) while keeping a safe gap to a slower lead car (CBF).
- "Adaptive": if the lead car is slower, you slow down even though your set speed is higher.

Step 1. Define your problem: Dynamics & Control Objectives.





State $x = [p \ v \ z]^T \in \mathbb{R}^3$

Control Input $u \in \mathbb{R}^1$

Dynamics

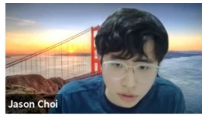
$$\dot{x} = \underbrace{\begin{bmatrix} v \\ -\frac{1}{m}F_r(v) \\ v_0 - v \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \\ 0 \end{bmatrix}}_{g(x)} u$$

$F_r(v) = f_0 + f_1 v + f_2 v^2$ is a rolling resistance.

Input constraints	$-mc_d g \le u \le mc_a g$
Stability objective	$v \to v_d$ (v_d: desired velocity)
Safety objective	$z \ge T_h v$ (T_h: lookahead time)

Step 1: model and objectives

- State $x = [p, v, z]$: position, speed, gap to lead car. Input $u =$ wheel force. Lead car speed v_0 is constant.
- $\dot{p} = v$, $\dot{v} = (u - F_r(v))/m$, $\dot{z} = v_0 - v$, with rolling resistance $F_r = f_0 + f_1 v + f_2 v^2$. Control affine, constant g.
- Input limits: $-c_d mg \le u \le c_a mg$ (max braking / max acceleration).
- Objectives: speed $v \to v_d$ (CLF); gap $z \ge T_h v$ (CBF), $T_h =$ lookahead time. Example: at 20 m/s with $T_h = 1.8$ s, keep at least 36 m.



Code: Create a class that inherit CtrlAffineSys.

dynsys/@ACC/ACC.m

```
classdef ACC < CtrlAffineSys
  methods
    % Constructor
    function obj = ACC(params)
      obj = obj@CtrlAffineSys(params);
      %% Add your own code here.
    end
    % Custom function (rolling resistance)
    function Fr = getFr(obj, x)
      v = x(2);
      Fr = obj.params.f0 + obj.params.f1 * v + obj.params.f2 * v^2;
    end
  end
end
```

- Create a class directory (e.g. '@ACC') and define your class. To use the built-in functions of the library, it should inherit the CtrlAffineSys class.

Code: the ACC class

- Skipped in the talk (on screen under a second); summary from the slide.
- File `dynsys/@ACC/ACC.m`: a MATLAB class folder `@ACC` whose class inherits the library's `CtrlAffineSys`, so all the built-in controllers work with it.
- The constructor just hands `params` to the parent class. You can add your own helpers, e.g. `getFr(obj, x)` returns the rolling resistance $F_r = f_0 + f_1 v + f_2 v^2$.

Code: Create a class function defineSystem and define your dynamics.



dynsys/@ACC/defineSystem.m

```
% Use the same input and output argument structure.
function [x, f, g] = defineSystem(~, params)
    syms p v z % states
    x = [p; v; z];                          <-- state x

    f0 = params.f0;
    f1 = params.f1;
    f2 = params.f2;
    v0 = params.v0;
    m = params.m;
    Fr = f0 + f1 * v + f2 * v^2;

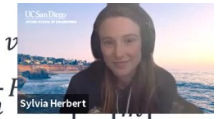
    % Dynamics
    f = [v; -Fr/m; v0-v];                   <-- f(x)
    g = [0; 1/m; 0];                        <-- g(x)
end
```

- **Use symbolic expression to write the dynamics.**
- **Create each vector fields $f(x)$ and $g(x)$ separately.**

Code: defineSystem

- Skipped in the talk; summary from the slide.
- `defineSystem(~, params)` returns the state x and the vector fields f, g as *symbolic* expressions (MATLAB `syms`).
- For ACC: $x = [p;\ v;\ z]$, $f = [v;\ -F_r/m;\ v_0 - v]$, $g = [0;\ 1/m;\ 0]$, the same model as Step 1.
- Write $f(x)$ and $g(x)$ separately: the library needs them apart to form the L_f and L_g terms.

Step 2. Design a CLF and evaluate.



$$\dot{x} = \underbrace{\begin{bmatrix} v \\ -\frac{1}{m}F_r \\ v_0 - v \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \\ 0 \end{bmatrix}}_{g(x)} u$$

Stability objective	$v \to v_d$ (v_d: desired velocity)

A Lyapunov function should be 0 at x_e($[\cdot \;\; v_d \;\; \cdot]^T$) and positive everywhere else.

Let's try the most intuitive one: $V(x) = (v - v_d)^2$

$$\nabla V(x) = [0 \;\; 2(v - v_d) \;\; 0]$$

$$L_f V(x) = -\frac{2}{m} F_r(v)(v - v_d),\; L_g V(x) = \frac{2}{m}(v - v_d)$$

Constraint: $\dot{V}(x,u) + \lambda V(x) = L_f V(x) + L_g V(x)u + \lambda V(x) = (v - v_d)\left\{\frac{2}{m}(u - F_r) + \lambda(v - v_d)\right\} \leq 0$

The value of rolling resistance F_r is very small.

If $v < v_d$, if we accelerate sufficiently large, $\left\{\frac{2}{m}(u - F_r) + \lambda(v - v_d)\right\}$ will be positive and we can make the constraint satisfied. Similarly, the constraint is satisfied for the opposite case ($v > v_d$) if we decelerate enough. Therefore, we conclude that this is a valid CLF.

Remark: If the value of λ becomes too large, depending on the input constraints the CLF can be invalid.

Step 2: design the CLF

- Try $V = (v - v_d)^2$. Then $\dot{V} + \lambda V = (v - v_d)\{\frac{2}{m}(u - F_r) + \lambda(v - v_d)\} \leq 0$ is the constraint.
- Too slow: accelerate hard enough and it holds. Too fast: brake hard enough and it holds. So V is a valid CLF.
- Q&A: with input limits, a large λ can make it *invalid*, because fast decay needs more force than the car has.
- In practice: linearize near the equilibrium and take a quadratic V from linear theory, or apply feedback linearization first and design V on the now-linear output dynamics.



Code: Create class functions defineClf and define your CLF.

dynsys/@ACC/defineClf.m

```
% Use the same input and output argument structure.
function clf = defineClf(obj, params, symbolic_state)
    v = symbolic_state(2);
    vd = params.vd; % desired velocity.

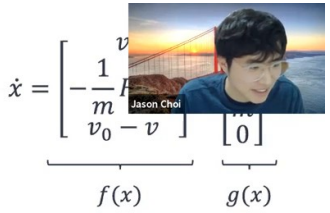
    clf = (v - vd)^2;  <-- V(x)
end
```

- Consider symbolic_state as the same vector s in symbolic expression that you defined in your defineSystem.
- Define your CLF using this vector and parameters.
- After setting up your defineClf, the class instance will hold function_handle obj.clf(x), obj.lf_clf(x), and obj.lg_clf(x) which are generated from these symbolic expressions.

Code: defineClf

- Skipped in the talk; summary from the slide.
- `defineClf(obj, params, symbolic_state)` receives the same symbolic state as `defineSystem` and returns $V(x)$. For ACC: `clf = (v - vd)^2`, the CLF from Step 2.
- The library then generates function handles `obj.clf(x)`, `obj.lf_clf(x)`, `obj.lg_clf(x)`, i.e. V, $L_f V$, $L_g V$. No differentiating by hand.

Step 3. Design a CBF and evaluate.



$$\dot{x} = \underbrace{\begin{bmatrix} v \\ -\frac{1}{m}F_r(v) \\ v_0 - v \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \\ 0 \end{bmatrix}}_{g(x)}$$

Safety objective	$z \geq T_h v$ (T_h: lookahead time)

- CBF $B(x)$ should be 0 at the "boundary", and positive in the safe region.
- Let's start with an intuitive choice for the CBF—$B(x) = z - T_h v$

$$L_f B(x) = \frac{T_h}{m}F_r(v) + (v_0 - v),\ L_g B(x) = -\frac{T_h}{m}$$

Constraint: $\dot{B}(x,u) + \gamma B(x) = \frac{T_h}{m}(F_r(v) - u) + (v_0 - v) + \gamma(z - T_h v) \leq 0$

Neglecting the effect of F_r, if we apply the maximum deceleration $u = -c_d m g$,

$$\dot{B}(x,u) + \gamma B(x) = T_h c_d g + v_0 + \gamma z - (1 + T_h \gamma) v$$

When the value of v is big compared to the positive terms (determined by c_d and v_0), the constraint might still not be satisfied.

To remedy this, we modify a CBF to include a term regarding a minimum braking distance required to decelerate from v to v_0:

$$B(x) = z - T_h v - \frac{1}{2}\frac{(v - v_0)^2}{c_d g}$$

Then we get $\dot{B}(x,u) = \frac{1}{m}\left(T_h + \frac{v - v_0}{c_d g}\right)(F_r(v) - u) + (v_0 - v)$.

Under maximum deceleration ($u = -c_d m g$), $\dot{B}(x,u) = \frac{1}{m}T_h F_r(v) + T_h c_d g > 0$.

Therefore, the constraint is always feasible at any state, so $B(x)$ is now a valid CBF.

Step 3: design the CBF

- First try $B = z - T_h v$. At high speed, even max braking can't satisfy $\dot{B} + \gamma B \geq 0$, so it is *not* a valid CBF.
- Fix: subtract the distance needed to brake down to v_0: $B = z - T_h v - \frac{1}{2}\frac{(v - v_0)^2}{c_d g}$.
- Under max braking, $\dot{B} = \frac{1}{m}T_h F_r + T_h c_d g > 0$, so the constraint can always be met. Now B is valid.
- Lesson: building the input limits into B is what makes it valid.

Code: Create class functions defineCbf and define your CBF!



dynsys/@ACC/defineCbf.m

```
% Use the same input and output argument structure.
function cbf = defineCbf(obj, params, symbolic_state)
    v = symbolic_state(2);
    z = symbolic_state(3);

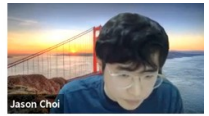
    v0 = params.v0;
    T = params.T;
    cd = params.cd;

    cbf = z - T * v - 0.5  * (v0-v)^2 / (cd * params.g);  <-- B(x)
end
```

- After setting up your defineCbf, the class instance will hold function_handle obj.cbf(x), obj.lf_cbf(x), and obj.lg_cbf(x) which are generated from these symbolic expressions.

Code: defineCbf

- Skipped in the talk; summary from the slide.
- `defineCbf` works the same way and returns $B(x)$. For ACC: `z - T*v - 0.5*(v0-v)^2/(cd*g)`, the braking-distance CBF from Step 3.
- The library generates `obj.cbf(x)`, `obj.lf_cbf(x)`, `obj.lg_cbf(x)` (B, L_fB, L_gB) for the QP's safety constraint.



Step 4. Implement and tune the parameters.

- **Once you got the valid CLF and CBF, implement your controller and run the simulation.**
- **Depending on your hyperparameters λ and γ, the QP can become infeasible. Also, the trajectories will depend a lot on the parameters. Therefore, be sure to tune these values according to the desired goals. (More details about this in https://github.com/HybridRobotics/CBF-CLF-Helper/blob/master/Manual_v1.pdf)**

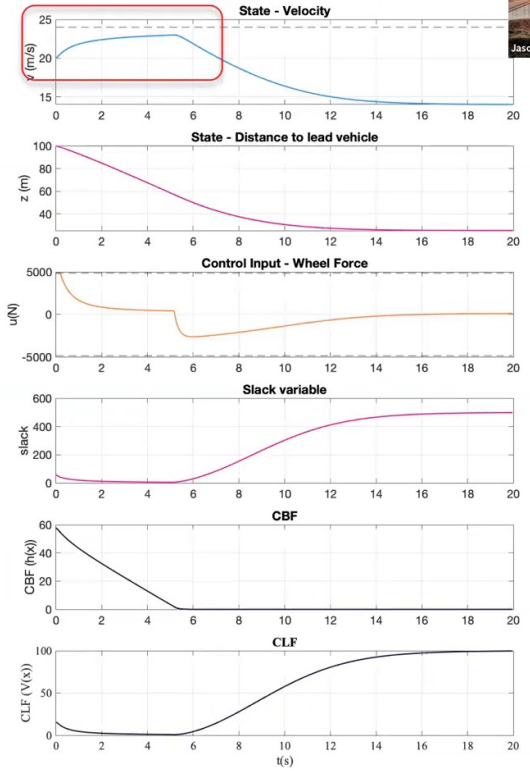
Step 4: implement and tune

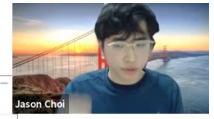
- Implement the QP controller and simulate.
- λ and γ change the behavior a lot, and bad values make the QP infeasible. Tune them for your goals (details in the repo's `Manual_v1.pdf`).

Results

1.The vehicle tries to reach the target velocity at the beginning (~5.2 sec).

2. As vehicle gets closer to the lead vehicle, because of the CBF constraint, it starts to decelerate.
   - Note that the CLF constraint is actively relaxed to give priority to the CBF constraint.

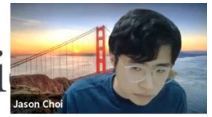




Results (1): speeding up

- Start: 20 m/s, 100 m behind a lead car doing 14 m/s; target speed 24 m/s.
- First ∼5.2 s: far from the lead car and below target, so the CLF accelerates toward 24 m/s and V falls smoothly toward 0.

Code: Create a class instance with parameters specifi as a Matlab structure array.



demos/run_cbf_clf_simulation_acc.

```
dt = 0.02; sim_t = 20; % sampling rate and terminal time of the simulation.
x0 = [0; 20; 100]; % Initial state

%% Parameters are from
% Aaron Ames et al. Control Barrier Function based Quadratic Programs
% with Application to Adaptive Cruise Control, CDC 2014, Table 1.
params.v0 = 14; % lead vehicle velocity.
params.vd = 24; % desired velocity.
% model parameters
params.m  = 1650; params.g = 9.81;
params.f0 = 0.1; params.f1 = 5; params.f2 = 0.25;
params.ca = 0.3; params.cd = 0.3;
params.T = 1.8;
% input constraints
params.u_max = params.ca * params.m * params.g;
params.u_min  = -params.cd * params.m * params.g;
% clf & cbf constraint parameters
params.clf.rate = 5;          <- λ
params.cbf.rate = 5;          <- γ
% weight parameters
params.weight.input = 2/params.m^2;   <- H
params.weight.slack = 2e-2;           <- p

%% Create class instance.
accSys = ACC(params);
```

- **params** should contain all necessary values for the model and the controller.
- The three essential values **params** should contain are
  - **params.clf.rate** (λ in CLF constraint)
  - **params.cbf.rate** (γ in CBF constraint)
  - **params.weight.slack** (weight for slack variable in the QP problem.)

Code: set up parameters

- `run_cbf_clf_simulation_acc.m`: put every model and controller value in a `params` struct, then `accSys = ACC(params)`.
- $x_0 = [0; 20; 100]$, $v_0 = 14$, $v_d = 24$, $dt = 0.02$ s, 20 s run. Model values from Ames et al. CDC 2014.
- Key knobs: `params.clf.rate` $= \lambda$, `params.cbf.rate` $= \gamma$, `params.weight.slack` $= p$ (and `weight.input` $= H$).



Code: Use the built-in functions—dynamics and other controllers su
as ctrlCbfClfQp, ctrlClfQp to simulate the system.

demos/run_cbf_clf_simulation_acc.m (continued)

```
odeFun = @accSys.dynamics;
controller = @accSys.ctrlCbfClfQp;
odeSolver = @ode45;

total_k = ceil(sim_t / dt);
x = x0; t = 0;
% initialize traces.
xs = zeros(total_k, 3); ts = zeros(total_k, 1);
us = zeros(total_k-1, 1); hs = zeros(total_k-1, 1); Vs = zeros(total_k-1, 1);
xs(1, :) = x0'; ts(1) = t;
% Run simulation.
for k = 1:total_k-1
   % Determine control input.
   Fr = accSys.getFr(x);
   [u, slack, h, V] = controller(x, Fr);
   us(k, :) = u';   hs(k) = h;   Vs(k) = V;

   % Run one time step propagation.
   [ts_temp, xs_temp] = odeSolver(@(t, s) odeFun(t, s, u), [t t+dt], x);
   x = xs_temp(end, :)';

   xs(k+1, :) = x';
   ts(k+1) = ts_temp(end);
   t = t + dt;
end
```

$u_{ref} = F_r$

- **dynamics** returns the value of $\dot{x}$, which can be used in the matlab ode functions for simulation.
- **ctrlCbfClfQp** and other built-in controller functions take state as input arguments, and outputs the feedback control.

$$\underset{\substack{u\text{: control input}\\ \delta\text{: slack variable}}}{\operatorname{argmin}} \; (u-u_{ref})^T H(u-u_{ref}) + p\delta^2$$

$$\text{subject to: } L_fV(x) + L_gV(x)u + \lambda V(x) \le \delta \quad \text{CLF Constraint}$$
$$L_fB(x) + L_gB(x)u + \gamma B(x) \ge 0 \quad \text{CBF Constraint}$$
$$u \in U \quad \text{Input Constraint}$$

Berkeley UNIVERSITY OF CALIFORNIA

2021-02-11 13:32:05

Code: simulation loop

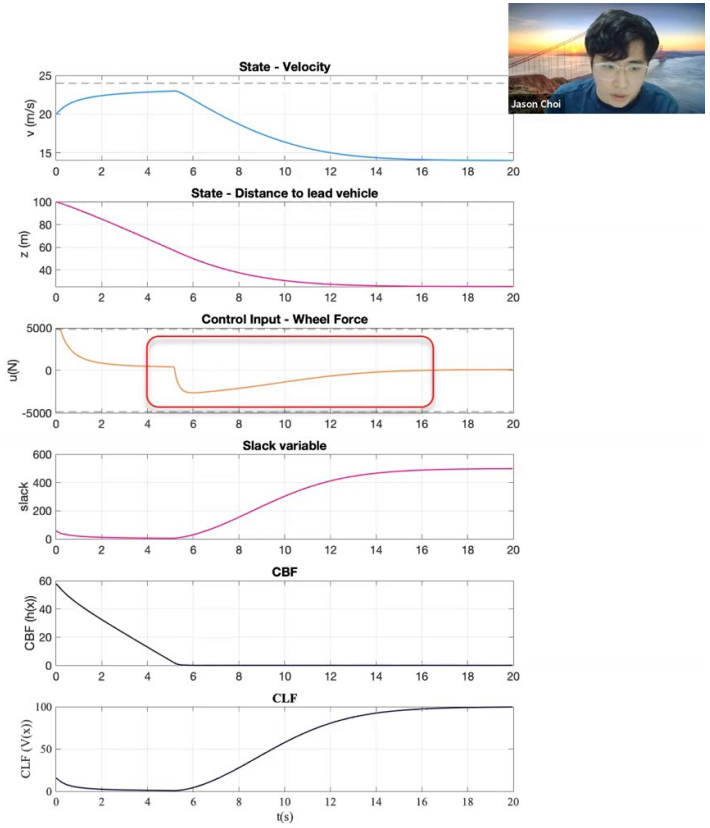
- Each step: `[u, slack, h, V] = controller(x, Fr)` with `ctrlCbfClfQp`, using $u_{\text{ref}} = F_r$; then integrate one step with `ode45`.
- `dynamics` returns $\dot{x}$; the built-in controllers take the state and return the feedback input.
- (Shown only for a few seconds in the talk.)

Results

1.The vehicle tries to reach the target velocity at the beginning (~5.2 sec).

2. As vehicle gets closer to the lead vehicle, because of the CBF constraint, it starts to decelerate.

- Note that the CLF constraint is actively relaxed to give priority to the CBF constraint.



Results (2): braking

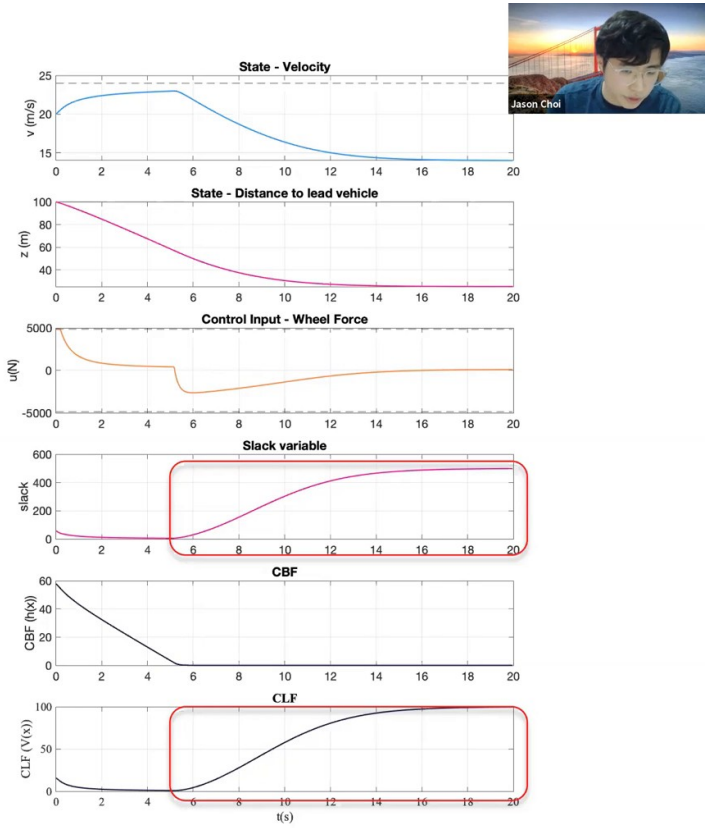
- As the gap shrinks, the CBF constraint becomes active and the car brakes: wheel force drops sharply at about 5.5 s.
- B falls to just above 0 and never crosses it. The car stays at the safe gap while its speed drops toward the lead car's 14 m/s.

Results

1.The vehicle tries to reach the target velocity at the beginning (~5.2 sec).

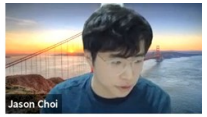
2. As vehicle gets closer to the lead vehicle, because of the CBF constraint, it starts to decelerate.

- **Note that the CLF constraint is actively relaxed to give priority to the CBF constraint.**



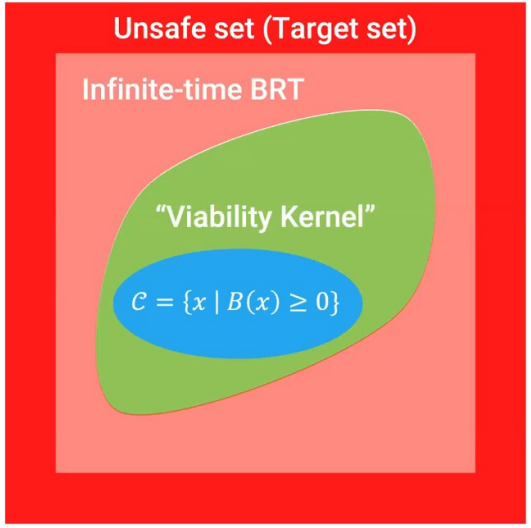
Results (3): CLF gives way

- Slack δ grows after $\sim$6 s: the CLF is relaxed, because the car wants 24 m/s but must slow down to stay safe.
- V rises to about $(14-24)^2 = 100$. That is expected: the QP puts safety ahead of tracking.



Comparison between CBF and HJ Reachability

Geometric connection



- **Viability Kernel: Maximum control invariant set that satisfies the safety constraint.**
  - **Complement of the infinite-time inevitable BRT.**
- **In other words, in the viability kernel, we can always find a control such that keeps the state safe.**
- **Every super-zero level set of the CBF we design for our safety objective should be control invariant set. Therefore, it is always contained in the viability kernel.**
- **Also, the value function of the viability kernel is a valid CBF.**

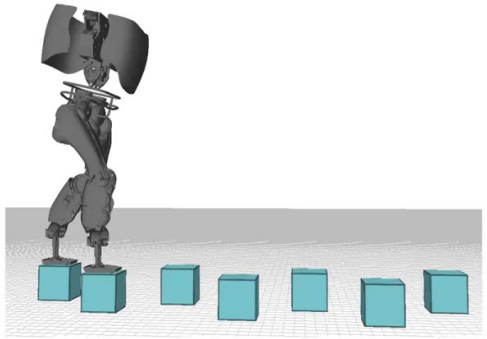
 ▶ 61:07 also ▶ 67:25

CBF vs HJ reachability: geometry

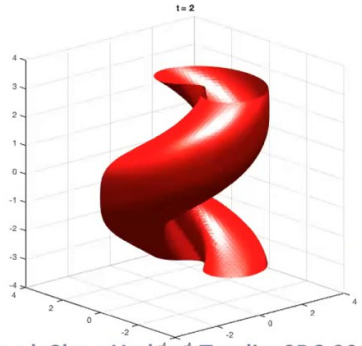
- HJ: mark the unsafe set and compute the infinite-time *inevitable* backward reachable tube (states that crash no matter what). Its complement is the **viability kernel**, the largest set where some input keeps you safe forever.
- Any CBF's safe set $\{B \geq 0\}$ must be control invariant, so it sits inside the viability kernel. It is often strictly smaller (conservative).
- The HJ value function of the viability kernel is itself a valid CBF, which gives one way to build a CBF.
- So why use a CBF-QP at all? See the next slide.



Comparison between CBF and HJ Reachability



[Nguyen et al., CDC 2016*]



[Bansal, Chen, Herbert, Tomlin, CDC 2017**]

Pros:

- Applicable to high-dimensional systems
- Applicable to real-time applications
- "Efficient"

Cons

- Not "optimal"
- Hard Verification / Construction

CBF

- Optimal
- Constructive

- Suffers curse of dimensionality
- The controller is "aggressive"

HJ Reachability

Berkeley UNIVERSITY OF CALIFORNIA

*3D dynamic walking on stepping stones with control barrier functions
** Hamilton-Jacobi reachability: A brief overview and recent advances

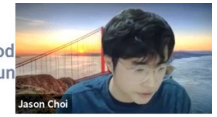
2021-02-11 13:40:23

CBF vs HJ: pros and cons

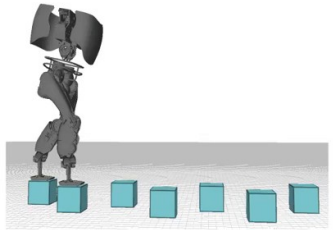
- **CBF:** scales to high-dimensional systems and runs in real time; the min-norm input saves energy. But it is not optimal for safety, and valid CBFs/CLFs are hard to build and verify (input limits, λ, γ).
- **HJ:** systematic construction and an optimal safety controller. But the curse of dimensionality limits grid solvers to about 5–6 states, and the control is aggressive (full brake / full throttle).
- Q&A: these CLF/CBFs bound the decay rate but don't promise arrival in finite time; finite- and prescribed-time versions are recent work. A poor u_{ref} raises the cost, but the constraints still hold.

Various Applications

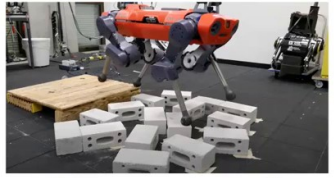
1. 3D dynamic walking on stepping stones with control barrier functions
2. Multi-Layered Safety for Legged Robots via Control Barrier Functions and Mod
3. Safety-Critical Model Predictive Control with Discrete-Time Control Barrier Fun
4. Safe teleoperation of dynamic uavs through control barrier functions
5. Safety Barrier Certificates for Collisions-Free Multirobot Systems
6. Safe Certificate-Based Maneuvers for Teams of Quadrotors Using Differential Flatness
7. Safety-Critical Control of Compartmental Epidemiological Models with Measurement Delays



Legged Robots

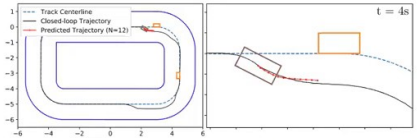


[Nguyen et al., CDC 2016[1] video]

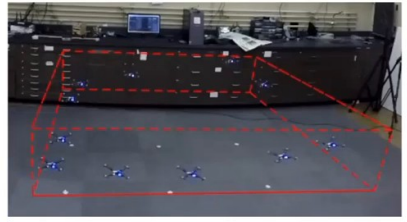


[Grandia et al., 2020[2]]

Autonomous mobile robots

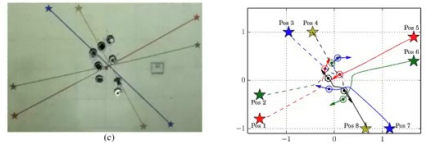


[Zeng et al., 2020[3]]



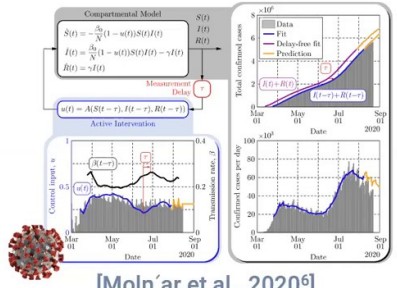
[Xu et al., ICRA 2018[4]]

Multi-agent systems



[Wang et al., TRo 2017[5] video, ICRA 2017[6]]

Infection control



[Moln´ar et al., 2020[6]]

Applications

- Legged robots (biped walking on stepping stones, quadrupeds), autonomous cars and drones, collision-free multi-robot teams, quadrotor swarms.
- Even epidemic control: keeping COVID infection levels within limits (Ames group, Caltech).

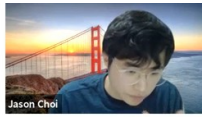


Devising new control designs

- **Discrete-time formulation**
  - Agrawal, Sreenath, Discrete Control Barrier Functions for Safety-Critical Control of Discrete Systems with Application to Bipedal Robot Navigation, RSS 2017.
- **Robust / adaptive control**
  - Nguyen, Sreenath, Optimal Robust Control for Bipedal Robots through Control Lyapunov Function based Quadratic Programs, RSS 2015.
  - Nguyen, Sreenath, L1 adaptive control for bipedal robots with control Lyapunov function based quadratic programs, ACC 2015.
  - Nguyen, Sreenath, Optimal Robust Control for Constrained Nonlinear Hybrid Systems with Application to Bipedal Locomotion, ACC 2016.
  - Jankovic, Robust control barrier functions for constrained stabilization of nonlinear systems, Automatica 2018.
  - Taylor, Ames, Adaptive safety with control barrier functions, ACC 2020.
  - Lopez, et al., Robust adaptive control barrier functions: An adaptive and data-driven approach to safety, L-CSS 2020.
- **Combining with optimal control, predictive control**
  - Zeng et al., Safety-Critical Model Predictive Control with Discrete-Time Control Barrier Function, 2020.
  - Grania et al., Nonlinear Model Predictive Control of Robotic Systems with Control Lyapunov Functions, RSS 2020.
  - Grania et al., Multi-Layered Safety for Legged Robots via Control Barrier Functions and Model Predictive Control, 2020.
  - Pereira et al., Safe Optimal Control Using Stochastic Barrier Functions and Deep Forward-Backward SDEs, CoRL2020.

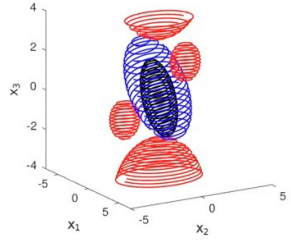
Research: new control designs

- Discrete-time CBFs for sampled-data systems.
- Robust and adaptive CLF/CBF-QPs that tolerate model errors.
- Combining with MPC (model predictive control) or even HJ, with CBF constraints inside those optimizers.



Theory

- Synthesizing a valid function
  - Sums-of-squares based methods
    - Tan, Packard, Searching for Control Lyapunov Functions using Sums of Squares Programming, 2004
    - Wang et al., Permissive barrier certificates for safe stabilization using sum-of-squares, ACC 2018
- Guaranteeing feasibility
  - How to design a valid CLF/CBF that CLF-/CBF-QP is feasible under the input constraints?
  - How to guarantee feasibility under multiple constraints?
    - Zeng et al. Safety-Critical Control with Optimal-decay Control Barrier Functions with Guaranteed Point-wise Feasibility, ACC 2021 (To be appeared)
- Connection to optimal control theory
  - Inverse Optimality
    - Freeman, Kokotovic, Inverse Optimality in Robust Stabilization, SIAM 1996.
    - Camilli, et al., Control Lyapunov functions and Zubov's method, SIAM 2008.
- Stochastic systems
    - Clark, Control barrier functions for stochastic systems, 2020.



[Wang et al., ACC 2018]

Research: theory

- Building valid CLFs/CBFs, e.g. sum-of-squares programming that searches for polynomial certificates.
- Feasibility: designing V and B so the QP is always solvable under input limits and many constraints, which is closely tied to validity. The speaker's group proposed optimal-decay CBFs with guaranteed pointwise feasibility (Zeng et al., ACC 2021).
- Inverse optimality: the QP controller is optimal for *some* cost, just not for safety. There are also CBFs for stochastic systems.



Combining with data-driven approaches: Dealing with uncertainties

- **Model Uncertainty**

Ground Vehicles



· Tire-slip dynamics

Aerial Vehicles



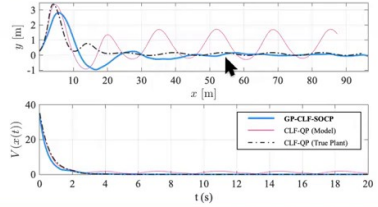
· Wind disturbance
· Ground effects

Legged Robots



· Kinematics & dynamics params.
· Actuator dynamics, Joint stiffness
· Ground contact
· Additional payload

- **Using Neural Networks**
  - Taylor et al., Learning for safety-critical control with control barrier functions, L4DC 2020
  - Choi, Castañeda et al., Reinforcement Learning for Safety-Critical Control under Model Uncertainty, using Control Lyapunov Functions and Control Barrier Functions (RL-CBF-CLF-QP), RSS 2020
- **Using Gaussian Process Regression**
  - Fan et al., Bayesian learning-based adaptive control for safety critical systems, 2019
  - Cheng et al., Safe multi-agent interaction through robust control barrier functions with learned uncertainties, 2020
  - Khojasteh et al.,Probabilistic safety constraints for learned high relative degree system dynamics, L4DC 2020
  - Castañeda, Choi et al., Gaussian Process-based Min-norm Stabilizing Controller for Control-Affine Systems with Uncertain Input Effects (GP-CLF-SOCP), 2020



[GP-CLF-SOCP, 2020]

Data-driven: model uncertainty

- $\dot{V}$ and $\dot{B}$ use the model's f and g, so model errors (tire slip, wind, unknown payload) break the guarantees.
- Fix: learn the mismatch with neural nets or Gaussian processes (GP) and include it in the constraint.
- Speaker's example (GP-CLF-SOCP): a bicycle model that ignores friction oscillates under a plain CLF-QP; adding the learned GP correction removes the oscillation.

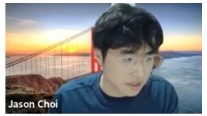


Combining with data-driven approaches: Dealing with uncertainties

- **Other kinds of uncertainties**
  - Uncertainty from the environment
    Uncertainty from the environment that is not captured by your dynamics.
    - Multi-agent
    - Human-robot interaction
    - Dynamic environment
  - Uncertainty in the constraints
    You don't exactly know what your safety constraint is.
  - Uncertainty from the measurement
    Uncertainty from imperfect and limited information about your system and environment.
    - Dean, Taylor, et al., Guaranteeing Safety of Learned Perception Modules via Measurement-Robust Control Barrier Functions, CoRL 2020.

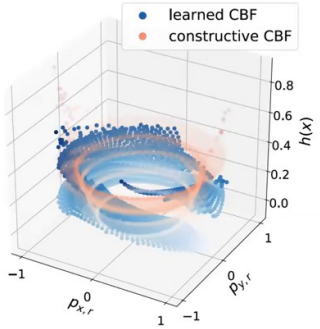
Data-driven: other uncertainties

- Environment: other agents, humans and moving obstacles that the dynamics model doesn't capture.
- Constraints: you don't know exactly where "unsafe" starts.
- Measurement: noisy or limited perception, e.g. measurement-robust CBFs for learned perception (Dean et al., CoRL 2020).



Combining with data-driven approaches: Synthesizing CLFs, CBFs with learning

- Huang et al., Feedback Stabilization Using Koopman Operator, CDC 2018.
- Richards et al., The Lyapunov Neural Network: Adaptive Stability Certification for Safe Learning of Dynamical Systems, CoRL 2018.
- Robey et al., Learning control barrier functions from expert demonstrations, CDC 2020.



[Robey et al., CDC 2020]

Data-driven: learning the functions

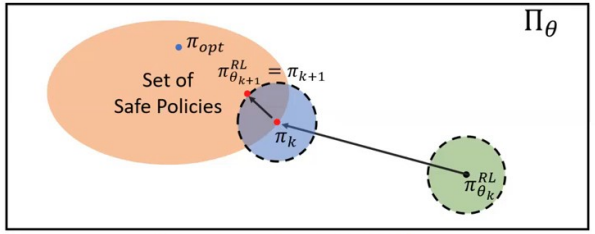
- Learn the CLF or CBF itself from data: Koopman operators, Lyapunov neural networks, CBFs learned from expert demonstrations.



Combining with data-driven approaches: Safe RL

Using Lyapunov theory or CBFs for Safe Reinforcement Learning

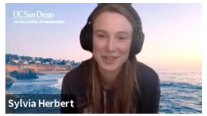
- Berkenkamp et al., Safe Model-based Reinforcement Learning with Stability Guarantees, NeurIPS 2017.
- Chow et al., A lyapunov-based approach to safe reinforcement learning, NeurIPS 2018.
- Cheng et al., End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks, AAAI 2019.
- Sun et al., FISAR: Forward Invariant Safe Reinforcement Learning with a Deep Neural Network-Based Optimizer, 2020.



[Cheng et al., AAAI 2019]

Data-driven: safe RL

- The RL community uses Lyapunov functions and CBFs to keep policies safe while learning, e.g. a CBF filter moves each RL action back into the safe set.



Thank you, Questions are welcomed!

jason.choi@berkeley.edu

CBF-CLF-Helper: https://github.com/HybridRobotics/CBF-CLF-Helper

Closing

- Code: CBF-CLF-Helper. Questions by email (address on slide).
- Post-talk chat: students comparing HJ reachability with CBFs in course projects, and labs comparing GP-based methods.